

ISIT312/ISIT912 Big Data Management

Spring 2025

MapReduce Practice

After this practice, you will get familiar with how to run MapReduce applications implemented in Python and using Hadoop Streaming.

Warning: To copy the fragments of code from this document to your working *Terminal* use *evince pdf viewer*.

Laboratory Instructions

Start a virtual machine `BigData-Ubuntu22.04.4-Hadoop3.3.6-P3-Spark3.5.1`

Note, that both the account name and the password are: `bigdata`

(0) Start Shell

After you log on the Ubuntu system with a user name `bigdata` and password `bigdata`, start a Terminal window with `Ctrl + Alt + T` or use the fifth from top icon located at a vertical stripe (sidebar) on the left-hand side of the screen.

The documents `LinuxCommandLineCheatSheet.pdf` and `Efficient-Linux-at-the-Command-Line-ch4.pdf` available through `Readings` link in `Resources` section in `linux` folder on Moodle contain more information on how to use Linux Shell in Terminal window.

You can use the Terminal window to interact with Hadoop. A simple hint that may make your life much easier is to use "Up" and "Down" keyboard keys to navigate through the commands already processed in Terminal window.

Now you can interact with Hadoop.

(1) Hadoop Initialisation

Now you can start Hadoop. Process a shell script `start-all.sh` in Terminal window in the following way:

```
$HADOOP_HOME/sbin/start-all.sh
```

The system will display few warnings and the following messages:

```
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bigdata-VirtualBox]
Starting resourcemanager
Starting nodemanagers
```

After you ignore the warning messages you can view the running processes with a command:

```
jps
```

The following results should be returned (note that the process numbers may be different):

```
819 NodeManager
2501 SecondaryNameNode
```

```
2310 DataNode
4391 Jps
2187 NameNode
2700 ResourceManager
```

(2) Run MapReduce applications

In the following, you will run some MapReduce applications in the Hadoop installation. Both applications are included in the `hadoop-mapreduce-examples-3.3.6.jar` file in a folder `$HADOOP_HOME/share/hadoop/mapreduce`.

This first application is the computation of Pi. You can start the applications by executing the following command:

```
$HADOOP_HOME/bin/hadoop jar \
$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar \
pi 10 20
```

Another application is searching the regular expressions beginning with the string `dfs`. First, create a folder `input` in a home folder of `bigdata` user.

```
$HADOOP_HOME/bin/hadoop fs -mkdir input
```

Then, copy all files located in a local file system folder `$HADOOP_HOME/etc/hadoop` to `input` folder

```
$HADOOP_HOME/bin/hadoop fs -put $HADOOP_HOME/etc/hadoop/* input
```

Next, remove a folder `shellprofile.d` and its contents from a folder `input`.

```
$HADOOP_HOME/bin/hadoop fs -rm input.shellprofile.d/*
$HADOOP_HOME/bin/hadoop fs -rmdir input.shellprofile.d
```

Note that the application will read from `input` folder in HDFS and it will write the results to `output` folder in HDFS. If the `output` folder already exists in HDFS, you need to remove it first. It means that if you run the application for the second time, the folder `output` must be removed first or you can use a different folder for the results.

Finally, run the application in the following way.:

```
$HADOOP_HOME/bin/hadoop jar \
$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar \
grep input output 'dfs[a-z.]+'
```

Check what are the outcomes of the application saved in `output` folder in HDFS.

```
$HADOOP_HOME/bin/hadoop fs -cat output/*
```

(3) Hadoop streaming with Linux shell executables and with mapper only

Selection

First, we use Hadoop streaming to filter data from a file located in HDFS. use a text editor (either `gedit` or `nano`) to create a file `marks.txt` where the first names are separated with `tab` character from the numbers. For a sample file see below.

```
alice    50
bruce    70
```

```
charlie 80
dan      75
```

Upload the file to HDFS in the following way.

```
hdfs dfs -put marks.txt
```

We would like to implement Hadoop streaming application that retrieves all lines in a file `marks.txt` that start from a letter `a` or from a letter `b`. We use Linux `grep` program with a regular expression to find the lines.

Process the following command in `Terminal` window to set up Linux variable `aorb` with invocation of `grep` program.

```
aorb='grep -E "^a|^b"'
```

Start Hadoop streaming application in the following way.

```
mapred streaming -D mapred.reduce.tasks=0 \
                  -input marks.txt \
                  -output output \
                  -mapper "$aorb"
```

A parameter `-D mapred.reduce.tasks=0` means that only a mapper is used. The results obtained from a mapper are saved in HDFS in a location determined by a parameter `-output output`. A location of input data in HDFS is determined by a parameter `-input marks.txt`. A mapper is `grep` program that selects the lines that match a regular expression `^a|^b`.

List the results with

```
hdfs dfs -cat output/p*
```

Remove the contents of `output` folder and the folder itself.

```
hdfs dfs -rm output/*
hdfs dfs -rmdir output
```

Transformation

The next application uses a mapper to transform the contents of an input file. We use a file `marks.txt` already uploaded to HDFS. An objective is to replace a lowercase `a` in a name `alice` with uppercase `A`. We use Linux stream editor `sed`.

Process the following command in `Terminal` window to set up Linux variable `sub` with invocation of `sed` program.

```
sub='sed s/al/Al/'
```

Next, start Hadoop streaming application in the following way.

```
mapred streaming -D mapred.reduce.tasks=0 \
                  -input marks.txt \
                  -output output \
                  -mapper "$sub"
```

A parameter `-D mapred.reduce.tasks=0` means that only a mapper is used. The results obtained from a mapper are saved in HDFS in a location determined by a parameter `-output output`. A location of input data in HDFS is determined by a parameter `-input marks.txt`. A mapper is `sed` program. The program replaces a string `a1` with `A1` in each line of input file.

List the results with

```
hdfs dfs -cat output/p*
```

Remove the contents of `output` folder and the folder itself.

```
hdfs dfs -rm output/*
hdfs dfs -rmdir output
```

In another example of transformation `cut` program is used to project an input file `marks.txt` on the second column, i.e. extract the second column from an input file `marks.txt`.

Process the following command in `Terminal` window to set up Linux variable `sub` with invocation of `sed` program.

```
take2nd='cut -f2'
```

Next, start Hadoop streaming application in the following way.

```
mapred streaming -D mapred.reduce.tasks=0 \
                  -input marks.txt \
                  -output output \
                  -mapper "$take2nd"
```

A parameter `-D mapred.reduce.tasks=0` means that only a mapper is used. The results obtained from a mapper are saved in HDFS in a location determined by a parameter `-output output`. A location of input data in HDFS is determined by a parameter `-input marks.txt`. A mapper is `cut` program. The program extracts the second column from an input file.

List the results with

```
hdfs dfs -cat output/p*
```

Remove the contents of `output` folder and the folder itself.

```
hdfs dfs -rm output/*
hdfs dfs -rmdir output
```

Pipelining

The next application pipelines the results obtained from a mapper to the next mapper. The first mapper extracts the second column from a file and the second mapper replaces `50` with `60` in the results obtained from the first mapper.

We use a file `marks.txt` already uploaded to HDFS. First, we use Linux stream editor `sed`.

Process the following command in `Terminal` window to set up Linux variable `sub` with invocation of `sed` program.

```
sub='sed s/50/60/'
```

Next, start Hadoop streaming applications in the following way.

```
mapred streaming -D mapred.reduce.tasks=0 \  
-input marks.txt \  
-output output \  
-mapper "$take2nd"  
  
mapred streaming -D mapred.reduce.tasks=0 \  
-input output/p* \  
-output output1 \  
-mapper "$sub"
```

A parameter `-D mapred.reduce.tasks=0` means that only a mapper is used. The results obtained from the first mapper are saved in HDFS in a location determined by a parameter `-output output`. A location of input data in HDFS is determined by a parameter `-input marks.txt`. A mapper is cut program. The second mapper takes input from the results created by the first mapper `output/p*` and saves its results in a folder `output1`. The second mapper replaces a string 50 with 60 in each line of input file.

List the results with

```
hdfs dfs -cat output1/p*
```

Remove the contents of the folders with the results and the folders themselves.

```
hdfs dfs -rm output/*  
hdfs dfs -rmdir output  
hdfs dfs -rm output1/*  
hdfs dfs -rmdir output1
```

(4) Hadoop streaming with Linux shell executables and with mapper and reducer

Now, we implement Hadoop streaming application that counts the total number of characters, total number of words and total number of lines in a text file. An application consists of both mapper and reducer.

A mapper is a Linux `cat` command that reads the files sequentially and it displays the contents of the files provides as arguments on a standard output (terminal).

A reducer is a Linux command line tool `wc` (word count) that finds the total number of newlines count, the total number of words count and the total number of characters count in the files provided as arguments.

Use a text editor to create a file `words.txt` with the following two lines.

```
word1 word2  
word3
```

Upload the file to HDFS into a home folder of bigdata user `/user/bigdata`. The following `hdfs` command will do.

```
hdfs dfs -put words.txt
```

Next, process the following `mapred` command.

```
mapred streaming \  
-input words.txt \  
-output output \  
-mapper /bin/cat \  
-reducer /usr/bin/wc
```

Hadoop streaming runs `cat` command to read a file `words.txt` located in HDFS. The results from `cat` are converted into key-value pairs where each line from `words.txt` is a key followed by tab character (`\t`) and an empty value. It means that end of line character (`\n`) immediately follows each tab (`\t`). Then, such key-value pairs are processed `wc` command that write the results into a file in `output` folder in HDFS.

List the results from processing with

```
hdfs dfs -cat output/p*
```

The results are the following.

```
2      3      20
```

It means that a file `words.txt` uploaded to HDFS consists of 2 lines, 3 words and 20 characters. If we process `cat` and `wc` in a pipeline on a file `words.txt` stored in a local file system

```
cat words.txt | wc
```

then the results are the following.

```
2      3      18
```

Two extra characters reported by `mapred` are the tab (`\t`) characters added by a mapper at the end of each line to separate keys from values.

(5) Hadoop streaming with Python and with mapper only

In this section we use Python to implement Hadoop streaming application that filters a text file with numbers. The application consists of a mapper only.

Use a text editor to create a file `numbers.txt` contains a sequence of numbers with empty lines (you can create any sequence of numbers):

```
23  
56  
  
78  
23  
12
```

Next, upload the file to HDFS.

```
hdfs dfs -put numbers.txt
```

Use a text editor to create a file `filter.py` with the following Python program.

```
#!/usr/bin/env python3

import sys

for number in sys.stdin:
    if (number != "\n") and int(number) > 50:
        print(int(number))
```

Then, change user's access permission to a file `filter.py` with

```
chmod u+x filter.py
```

The program uses Hadoop streaming feature to read a file `filter.py` from HDFS, to filter out the numbers less or equal to 50 and to save the results in HDFS.

Make sure that your file `filter.py` is located in `/home/bigdata`. Then, process the program with Hadoop streaming feature in the following way.

```
mapred streaming \
-D mapred.reduce.tasks=0 \
-input numbers.txt \
-output output \
-mapper /home/bigdata/filter.py
```

Note a parameter `-D mapred.reduce.tasks=0`. It means that only a mapper is used. The results obtained from a mapper are saved in HDFS in a location determined by a parameter `-output output`. A location of input data in HDFS is determined by a parameter `-input numbers.txt`.

The results can be found in HDFS with

```
hdfs dfs -ls output
```

and later on, listed with

```
hdfs dfs -cat output/p*
```

Remove the contents of `output` folder and the folder itself.

```
hdfs dfs -rm output/*
hdfs dfs -rmdir output
```

(6) Hadoop streaming with Python and with mapper and reducer

In this section we use Python to implement a classical Hadoop streaming application that counts the total number of occurrences of each word in a text file. The application consists of a mapper and a reducer.

A mapper reads the lines from a file in HDFS when processed with `mapred streaming` command, split the lines into the words and writes the key-value pairs `word \t 1`.

Create a file `mapper.py` with the following contents.

```
#!/usr/bin/env python3

import sys

for line in sys.stdin:
    line = line.strip()
    words = line.split()
    for word in words:
        print ('%s\t%s' % (word, 1))
```

When ready, change your access rights to a file `mapper.py` with `chmod` command.

```
chmod u+x mapper.py
```

A reducer reads the key-value pairs created by a program `mapper.py` and counts the words.

Create a file `reducer.py` with the following contents.

```
#!/usr/bin/env python3

import sys

current_word = None
current_count = 0
word = None
for line in sys.stdin:
    line = line.strip()
    word, count = line.split('\t', 1)
    try:
        count = int(count)
    except ValueError:
        continue
    if current_word == word:
        current_count += count
    else:
        if current_word:
            print ('%s\t%s' % (current_word, current_count))
            current_count = count
            current_word = word
        if current_word == word:
            print ('%s\t%s' % (current_word, current_count))
```

When ready, change your access rights to a file `reducer.py` with `chmod` command.

```
chmod u+x reducer.py
```

Insert few more words to a file `words.txt` and replace the file in HDFS with a new version.

Make sure that a folder `/home/bigdata` contains the files `mapper.py` and `reducer.py`. Next, process the program with Hadoop streaming feature in the following way.

```
mapred streaming \
-input words.txt \
-output output \
```

```
-mapper /home/bigdata/mapper.py \  
-reducer /home/bigdata/reducer.py
```

The results can be found in HDFS with

```
hdfs dfs -ls output
```

and later on, listed with

```
hdfs dfs -cat output/p*
```

Remove the contents of `output` folder and the folder itself.

```
hdfs dfs -rm output/*  
hdfs dfs -rmdir output
```

(7) Hadoop streaming with Python and with mapper and reducer

In this section we use Python to implement an application that processes a well formatted text file with the sample orders and provides the simple statistics. The statistics include information about the total number of orders and total number of items ordered per each item. The application consists of a mapper and a reducer.

A text file contains information about the orders submitted by the customers. A sample contents of the file is given below. Each line contains information about a single order. it consists of item name and the total number of items ordered.

```
bolt 50  
bolt 100  
nut 20  
bolt 100  
nail 30  
nut 40
```

Create a file `orders.txt` with the contents like above.

Upload the file to HDFS.

```
hdfs dfs -put orders.txt
```

A mapper reads the lines from a file in HDFS when processed with `mapred streaming` command, split the lines into key-value pairs: `item-name \t total-number-of items`.

Create a file `items.py` with the following contents.

```
#!/usr/bin/env python3  
  
import sys  
  
for line in sys.stdin:  
    line = line.strip()  
    item, total = line.split(' ', 1)  
    print ('%s\t%s' % (item, total))
```

When ready, change your access rights to a file `items.py` with `chmod` command.

```
chmod u+x items.py
```

A reducer reads the key-value pairs created by a program `items.py`, counts the orders per each item and summarizes the items per each item.

Create a file `stats.py` with the following contents.

```
#!/usr/bin/env python3

import sys

current_item = None
current_total = 0
current_counter = 1
item = None

for line in sys.stdin:
    line = line.strip()
    item, total = line.split('\t', 1)
    total = int(total)
    if current_item == item:
        current_total += total
        current_counter += 1
    else:
        if current_item:
            print('%s\t%s\t%s' % (current_item, current_total, current_counter))
            current_total = total
            current_item = item
            current_counter = 1
        if current_item == item:
            print('%s\t%s\t%s' % (current_item, current_total, current_counter))
```

When ready, change your access rights to a file `stats.py` with `chmod` command.

```
chmod u+x stats.py
```

Make sure that a folder `/home/bigdata` contains the files `items.py` and `stats.py`. Next, process the program with Hadoop streaming feature in the following way.

```
mapred streaming \
  -input orders.txt \
  -output output \
  -mapper /home/bigdata/items.py \
  -reducer /home/bigdata/stats.py
```

The results can be found in HDFS with

```
hdfs dfs -ls output
```

and later on, listed with

```
hdfs dfs -cat output/p*
```

Remove the contents of `output` folder and the folder itself.

```
hdfs dfs -rm output/*
hdfs dfs -rmdir output
```

(8) Hadoop streaming with Python and with mapper and reducer

In this section we use modern Python to implement an application that a classical Hadoop streaming application that counts the total number of occurrences of each word in a text file. The application consists of the following mapper

```
#!/usr/bin/env python3

import sys

def read_input(file):
    for line in file:
        yield line.split()

def main(separator='\t'):
    data = read_input(sys.stdin)
    for words in data:
        for word in words:
            print ('%s%s%d' % (word, separator, 1))

if __name__ == "__main__":
    main()
```

and the following reducer.

```
#!/usr/bin/env python3

from itertools import groupby
from operator import itemgetter
import sys

def read_mapper_output(file, separator='\t'):
    for line in file:
        yield line.rstrip().split(separator, 1)

def main(separator='\t'):
    data = read_mapper_output(sys.stdin, separator=separator)
    for current_word, group in groupby(data, itemgetter(0)):
        try:
            total_count = sum(int(count) for current_word, count in group)
            print ("%s%s%d" % (current_word, separator, total_count))
        except ValueError:
            pass

if __name__ == "__main__":
    main()
```

Save a mapper in a file `mmapper.py` and a reducer in a file `mreducer.py`. When ready, change your access rights to both files with `chmod` command.

```
chmod u+x m*.py
```

Make sure that a folder `/home/bigdata` contains the files `mmapper.py` and `mreducer.py`. Next, process the program with Hadoop streaming feature in the following way.

```
mapred streaming \
-input words.txt \
-output output \
-mapper /home/bigdata/mmapper.py \
-reducer /home/bigdata/mreducer.py
```

The results can be found in HDFS with

```
hdfs dfs -ls output
```

and later on, listed with

```
hdfs dfs -cat output/p*
```

Remove the contents of `output` folder and the folder itself.

```
hdfs dfs -rm output/*
hdfs dfs -rmdir output
```

(9) Hadoop streaming with Python and with mapper and reducer

In this section we use modern Python to implement an application that processes a well formatted text file with the sample orders and provides the simple statistics. The statistics include information about the total number of orders and total number of items ordered per each item. The application consists of a mapper and a reducer.

We use a file `orders.txt` uploaded to HDFS in step (7).

A mapper reads the lines from a file in HDFS when processed with `mapred streaming` command, split the lines into key-value pairs: `item-name \t total-number-of items`.

Create a file `mitems.py` with the following contents.

```
#!/usr/bin/env python3

import sys

def read_input(file):
    for line in file:
        yield line.split()

def main(separator='\t'):
    data = read_input(sys.stdin)
    for orders in data:
        print ('%s%s%s' % (orders[0], separator, orders[1]))

if __name__ == "__main__":
    main()
```

When ready, change your access rights to a file `mitems.py` with `chmod` command.

```
chmod u+x mitems.py
```

A reducer reads the key-value pairs created by a program `mitems.py`, counts the orders per each item and summarizes the items per each item.

Create a file `mstats.py` with the following contents.

```
#!/usr/bin/env python3

from itertools import groupby
from operator import itemgetter
import sys

def read_mapper_output(file, separator='\t'):
    for line in file:
        yield line.rstrip().split(separator, 1)

def main(separator='\t'):
    data = read_mapper_output(sys.stdin, separator=separator)
    for current_item, group in groupby(data, itemgetter(0)):
        try:
            total_items = 0
            total_transactions = 0
```

```

        for current_item, total in group:
            total_items += int(total)
            total_transactions += 1
            print ("%s%s%d%s%d" % (current_item, separator, total_items, separator,
total_transactions))
        except ValueError:
            pass

if __name__ == "__main__":
    main()

```

When ready, change your access rights to a file `mstats.py` with `chmod` command.

```
chmod u+x mstats.py
```

Make sure that a folder `/home/bigdata` contains the files `mitems.py` and `mstats.py`. Next, process the program with Hadoop streaming feature in the following way.

```

mapred streaming \
-input orders.txt \
-output output \
-mapper /home/bigdata/mitems.py \
-reducer /home/bigdata/mstats.py

```

The results can be found in HDFS with

```
hdfs dfs -ls output
```

and later on, listed with

```
hdfs dfs -cat output/p*
```

Remove the contents of `output` folder and the folder itself.

```

hdfs dfs -rm output/*
hdfs dfs -rmdir output

```
